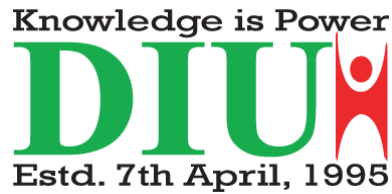


Dhaka International university



Lab Report

Lab Report Topic : Contiguous Memory Allocation (First Fit, Best Fit, Worst Fit)

Lab Report No : 05

Course Title : Operating system lab

Course Code :0613-301

Submitted By:	Submitted To:
Md.Yeasin Bhuiyan Roll:43 Reg No : CS-D-91-23-124699	Md. Nazmus sakib, Lecturer of CSE Department

Date of Submission:17-01-2026

1.Objectives:

- To understand contiguous memory allocation
- To implement First Fit, Best Fit and Worst Fit algorithms using C
- To compare memory fragmentation

2. Background Study (Short):

Contiguous memory allocation allocates a single continuous block of memory to a process. When multiple free blocks are available, the operating system uses different strategies to choose a block.

- First Fit: Allocates the first block that is large enough.
- Best Fit: Allocates the smallest suitable block.
- Worst Fit: Allocates the largest available block.

Example:

Job Size Block Sizes Available

Job1 90k 50k, 100k, 90k, 200k, 50k

Job2 20k

Job3 50k

Job4 200k

- First Fit: Job1 → Block2, Job2 → Block1, Job3 → Block3, Job4 → Block4
- Best Fit: Job1 → Block2, Job2 → Block5, Job3 → Block3, Job4 → Block4
- Worst Fit: Job1 → Block4, Job2 → Block4 remaining, Job3 → Block2, Job4 → Block4

3. Code

```
#include <stdio.h>
```

```
#define MAX 25
```

```
void firstFit(int block[], int m, int process[], int n) {
```

```
    int allocation[MAX], frag[MAX];
```

```
    for (int i = 0; i < n; i++) allocation[i] = -1;
```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (block[j] >= process[i]) {
            allocation[i] = j;
            frag[i] = block[j] - process[i];
            block[j] -= process[i];
            break;
        }
    }
}

printf("\nFirst Fit Allocation:\n");

printf("Process\tSize\tBlock\tFragment\n");

for (int i = 0; i < n; i++) {
    if (allocation[i] != -1)
        printf("%d\t%d\t%d\t%d\n", i+1, process[i], allocation[i]+1, frag[i]);
    else
        printf("%d\t%d\tNot Allocated\n", i+1, process[i]);
}
}

void bestFit(int block[], int m, int process[], int n) {
    int allocation[MAX], frag[MAX];

    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;

        for (int j = 0; j < m; j++) {
            if (block[j] >= process[i]) {
                if (bestIdx == -1 || block[j] < block[bestIdx]) {
                    bestIdx = j

```

```

    }
    }
}
if (bestIdx != -1) {
    allocation[i] = bestIdx;
    frag[i] = block[bestIdx] - process[i];
    block[bestIdx] -= process[i];
}
}

printf("\nBest Fit Allocation:\n");
printf("Process\tSize\tBlock\tFragment\n");
for (int i = 0; i < n; i++) {
    if (allocation[i] != -1)
        printf("%d\t%d\t%d\t%d\n", i+1, process[i], allocation[i]+1, frag[i]);
    else
        printf("%d\t%d\tNot Allocated\n", i+1, process[i]);
}
}

```

```

void worstFit(int block[], int m, int process[], int n) {
    int allocation[MAX], frag[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (block[j] >= process[i]) {
                if (worstIdx == -1 || block[j] > block[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            frag[i] = block[worstIdx] - process[i];
            block[worstIdx] -= process[i];
        }
    }
}

```

```

        }
    }
}

if (worstIdx != -1) {
    allocation[i] = worstIdx;
    frag[i] = block[worstIdx] - process[i];
    block[worstIdx] -= process[i];
}
}

printf("\nWorst Fit Allocation:\n");
printf("Process\tSize\tBlock\tFragment\n");
for (int i = 0; i < n; i++) {
    if (allocation[i] != -1)
        printf("%d\t%d\t%d\t%d\n", i+1, process[i], allocation[i]+1, frag[i]);
    else
        printf("%d\t%d\tNot Allocated\n", i+1, process[i]);
}
}

int main() {
    int block[MAX], process[MAX], m, n, choice;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    for (int i = 0; i < m; i++) {
        printf("Block %d size: ", i+1);
        scanf("%d", &block[i]);
    }
}

```

```

}

printf("Enter number of processes: ");

scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("Process %d size: ", i+1);
    scanf("%d", &process[i]);
}

int blockCopy[MAX];

while (1) {
    printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\n4. Exit\nEnter choice: ");
    scanf("%d", &choice);

    for (int i = 0; i < m; i++) blockCopy[i] = block[i];

    switch(choice) {
        case 1: firstFit(blockCopy, m, process, n); break;
        case 2: bestFit(blockCopy, m, process, n); break;
        case 3: worstFit(blockCopy, m, process, n); break;
        case 4: return 0;
        default: printf("Invalid choice!"); break;
    }
}

return 0;
}

```

4. Results

Input:

- Number of blocks: 5
- Block sizes: 50, 100, 90, 200, 50
- Number of processes: 4
- Process sizes: 90, 20, 50, 200

Output:

Algorithm	Process	Block	Fragment
First Fit	90	2	10
First Fit	20	1	30
First Fit	50	3	40
First Fit	200	4	0
Best Fit	90	3	0
Best Fit	20	5	30
Best Fit	50	2	50
Best Fit	200	4	0
Worst Fit	90	4	110
Worst Fit	20	4	90
Worst Fit	50	2	50
Worst Fit	200	4	-

5. Discussion

- First Fit: Fast allocation but may lead to fragmentation.
- Best Fit: Reduces wasted space but takes longer to find suitable block.
- Worst Fit: Leaves large blocks available but may increase fragmentation of smaller blocks.
- Each technique has advantages depending on system requirements.